Lecture 5

Space complexity

Algorithm Analysis: Time vs. Space Complexity

```
int x = 5;
int y = 7;
int z = x + y;
print(z);
if (z == 13) print("Done");
```

Focuses on executable lines/operations.

6 operations executed $\longrightarrow$ $T(n) = 6$

Result: O(1) (Constant Time)

Focuses **ONLY** on variable declarations.

$\rightarrow$ $S(n) = 3$

Result: O(1) (Constant Space)

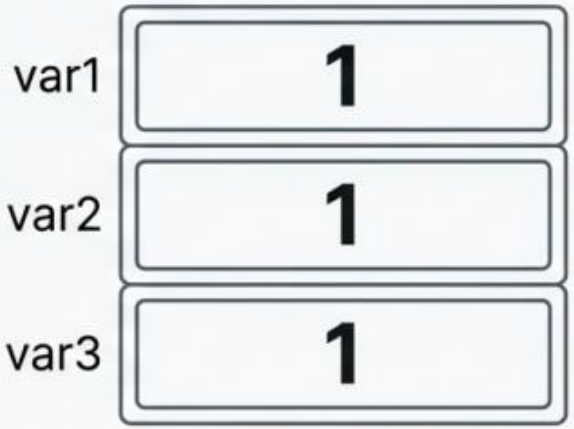
Time counts how many times lines execute. **Space** counts how many memory containers are created.

© 2026 Ahmed Elrefaiy

Simple Variable Allocation Yields Constant Space

```
int var1 = 10;
int var2 = 20;
int var3 = 30;
```

Memory Map



Total Memory $= 1 + 1 + 1$

Equation: $S(n) = 3$

Final Complexity: O(1)

Because the number of variables is fixed and does not scale with any input size (n), the space complexity remains constant.

Loops Do Not Multiply Space

```
int n = user_input;
for (int i = 0; i < n; i++) {
  // do something
}
```

Total Memory = 1 + 1

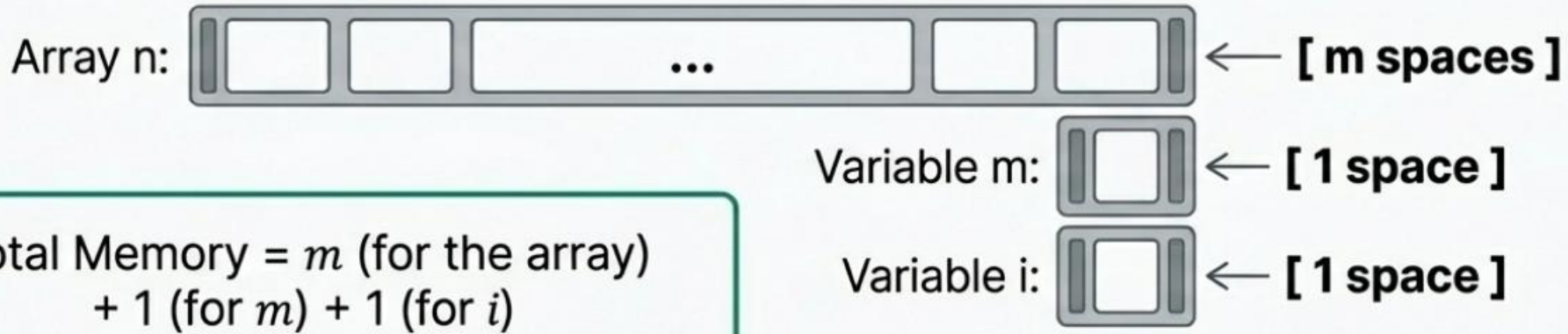
Equation: $S(n) = 2$

Final Complexity: O(1)

Crucial Insight
Wait, doesn't `i` change value?
Yes, but it uses the same single memory container throughout the entire loop. Changing an internal value does not require new memory space.

Arrays Introduce Linear Scaling

```
int m = user_input;
int[] n = new int[m];
for (int i = 1; i <= m; i++) { print(n[i]); }
```



Total Memory = m (for the array) + 1 (for m) + 1 (for i)

Equation: $S(m) = m + 2$

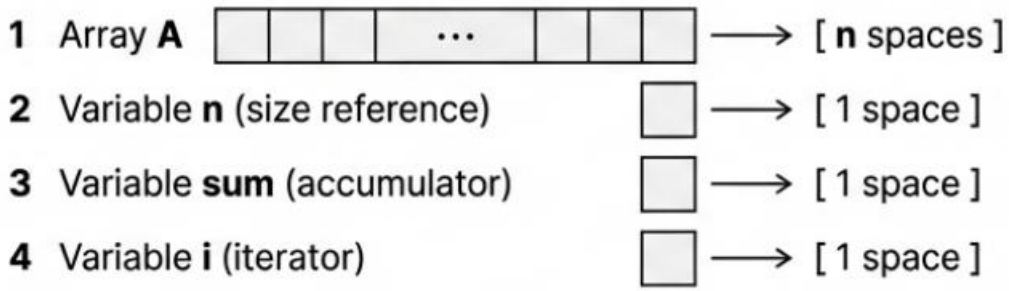
Final Complexity: O(m)

This is a first-degree (linear) equation. Because the array size depends directly on m, the space required scales linearly.

Comprehensive Linear Space: Arrays, Accumulators, and Iterators

```
int[] A = new int[n];
int sum = 0;
for (int i = 0; i < n; i++) {
  sum += A[i];
}
```

Memory Map Breakdown



Total Memory = n + 1 + 1 + 1

Equation: $S(n) = n + 3$

Final Complexity: **O(n)**

No matter how many times the loop runs or how high the sum gets, **sum**, **n**, and **i** each remain single containers. Only the array **A** expands memory needs based on input size.

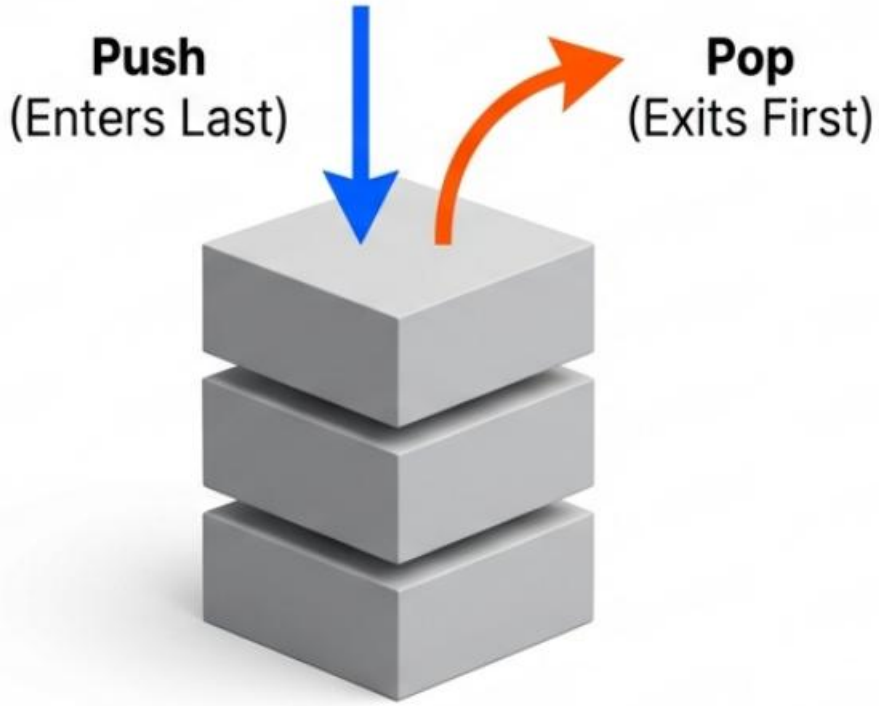
Space complexity is determined by the maximum simultaneous memory usage, not the total variable count.

The Memory Stack Lifecycle

- When a function is called, its variables are **stacked** in memory.
- When a function reaches its closing bracket, its variables are instantly **deleted**.

> Never just add all variables together. Walk through the code, take snapshots of the stack size, and find the **Max Peak**.

First In, Last Out (FILO / LIFO)



Nested functions stack variables sequentially until they resolve.

Coding Example

```
void f3() {
  int h = 4;
  // Snapshot 1 here
}

void f1() {
  int y = 7;
  f3();
}

void Main() {
  int x = 5;
  f1();
}
```

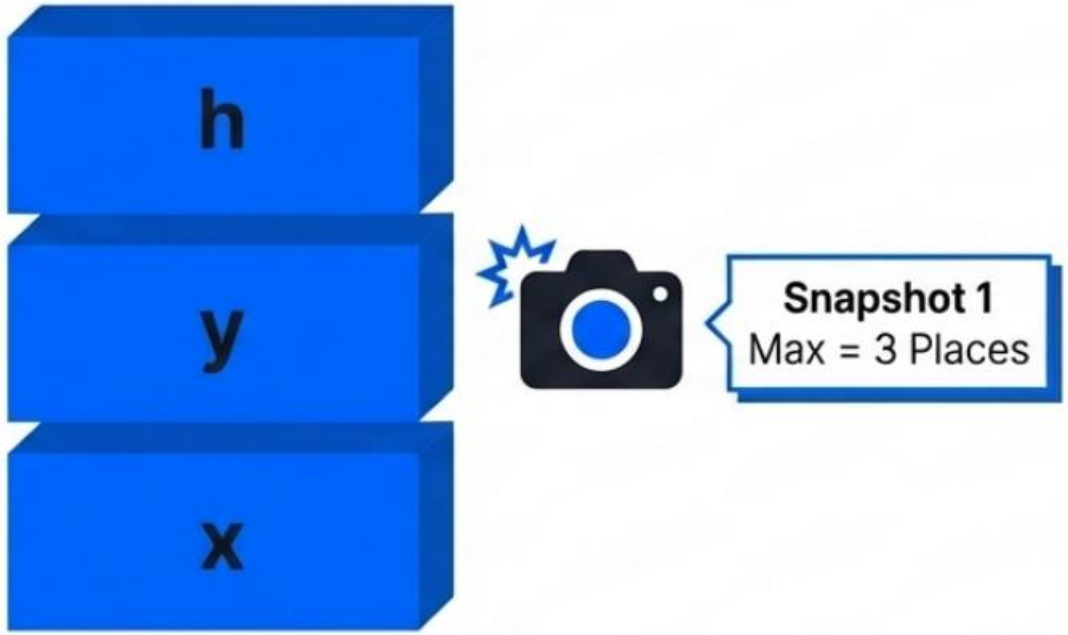
Execution Path Annotations

Step 1: Main is called. Variable x = 5 is stored. (Stack Size = 1)

Step 2: Main pauses. **f1** is called. Variable y = 7 is stored. (Stack Size = 2)

Step 3: f1 pauses. **f3** is called. Variable h = 4 is stored. (Stack Size = 3)

Memory Stack



All three variables exist in the memory stack simultaneously.

As functions finish, memory is released and recycled for new operations.

Execution Path Annotations

Step 4: f3 finishes. Removed **h**.

Step 5: f1 finishes. Removed **y**.
(Stack shrinks back to 1)

Step 6: Main resumes and calls **f2**.
Variable **z** = 11 is stored on top of **x**.

Memory Stack - The Descent

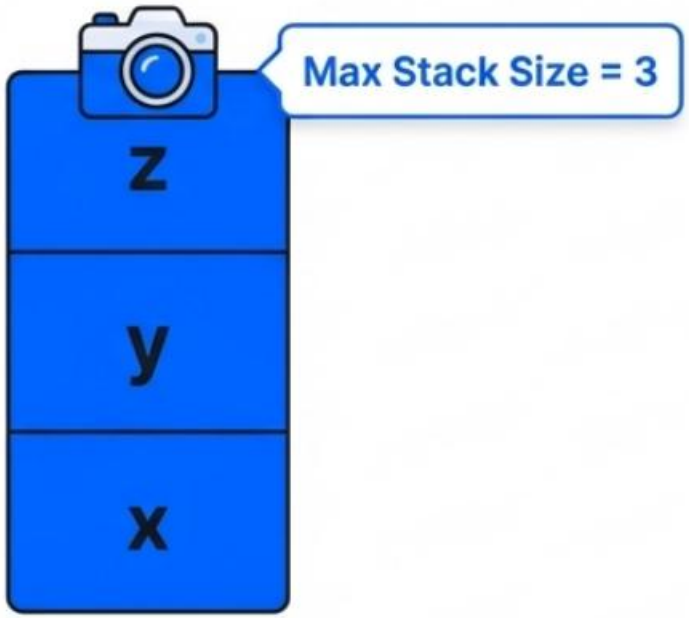
Comparing snapshots (3 vs. 2), the absolute highest memory usage was **3 variables**.

Space Complexity = 3 → O(1) Constant Space.

Standard sequential execution traps all scoped variables until the function completes.

```
void Main() {
  int x = 5;
  int y = 7;
  int z = x + y;
}
```

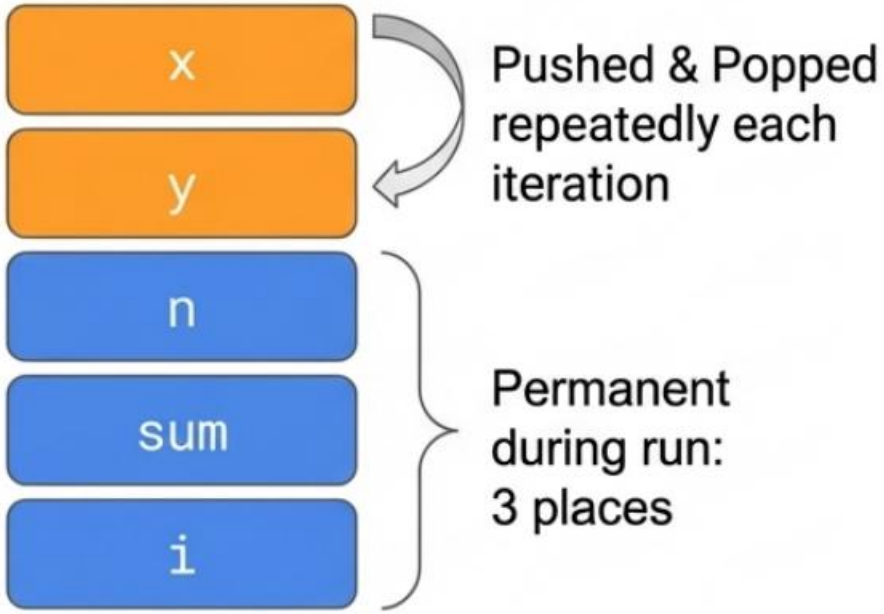
Because variables **x**, **y**, and **z** are **within the same scope**, none are released until Main hits the final closing bracket.



Space Complexity = 3 → O(1) Constant Space.

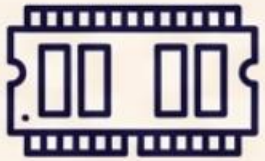
Loops reuse memory iteratively rather than expanding it infinitely.

```
def pair_sum(a, b):
    x = a * 2   # Nested scope variable
    y = b + 1   # Nested scope variable
    return x + y

def add_sequence(n):
    sum = 0             # Base scope variable
    for i in range(n): # Base scope variable
        sum += pair_sum(i, i + 1)
    return sum

# Example call
add_sequence(10)
```



The stack continuously expands to 5 and shrinks to 3. The absolute max snapshot is always exactly 5.
Space Complexity = 5 -> O(1) Constant Space.

The Golden Rule of Recursive Space Complexity



$$\text{Space Complexity} = \left[\text{Height of Recursion Tree}\right] \times \left[\text{Number of Variables per Call}\right]$$

The Misconception vs. Reality

- ❌ Space complexity is NOT the total number of function calls.
- ✅ It is the maximum memory used at any **single, concurrent moment** during execution.

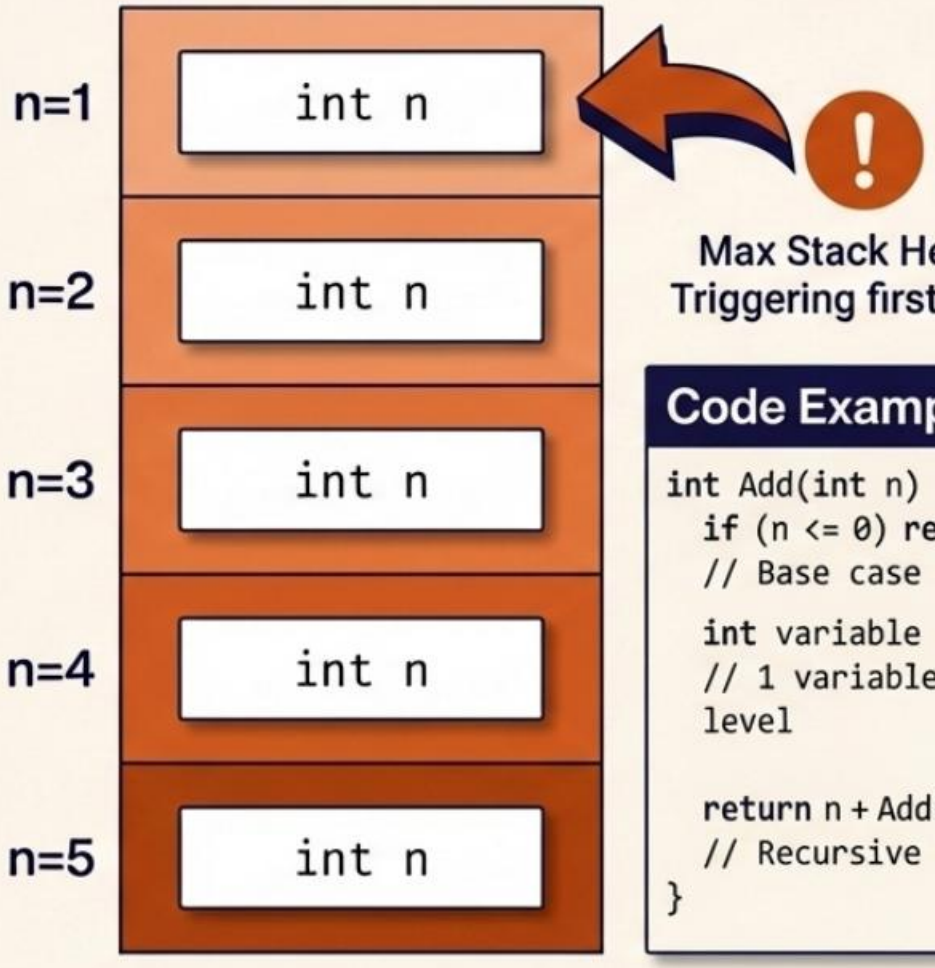
Defining the Factors

- **Variables**: How many new data points are stored in a single execution of the function? (e.g., declaring `int n = 1` variable).
- **Height**: What is the maximum depth the function reaches before hitting the base case (e.g., `n <= 0`) and returning?

Formula: Space Complexity = Height × Variables

NotebookLM

Linear Recursion: Building the Memory Stack



Max Stack Height:
Triggering first return

Code Example

```
int Add(int n) {
  if (n <= 0) return 0;
  // Base case

  int variable = n;
  // 1 variable per
  level

  return n + Add(n - 1);
  // Recursive call
}
```

1. The Scenario

Function Add(n) recursively calls Add(n-1) until reaching the base case n=0.

2. The Memory Reality

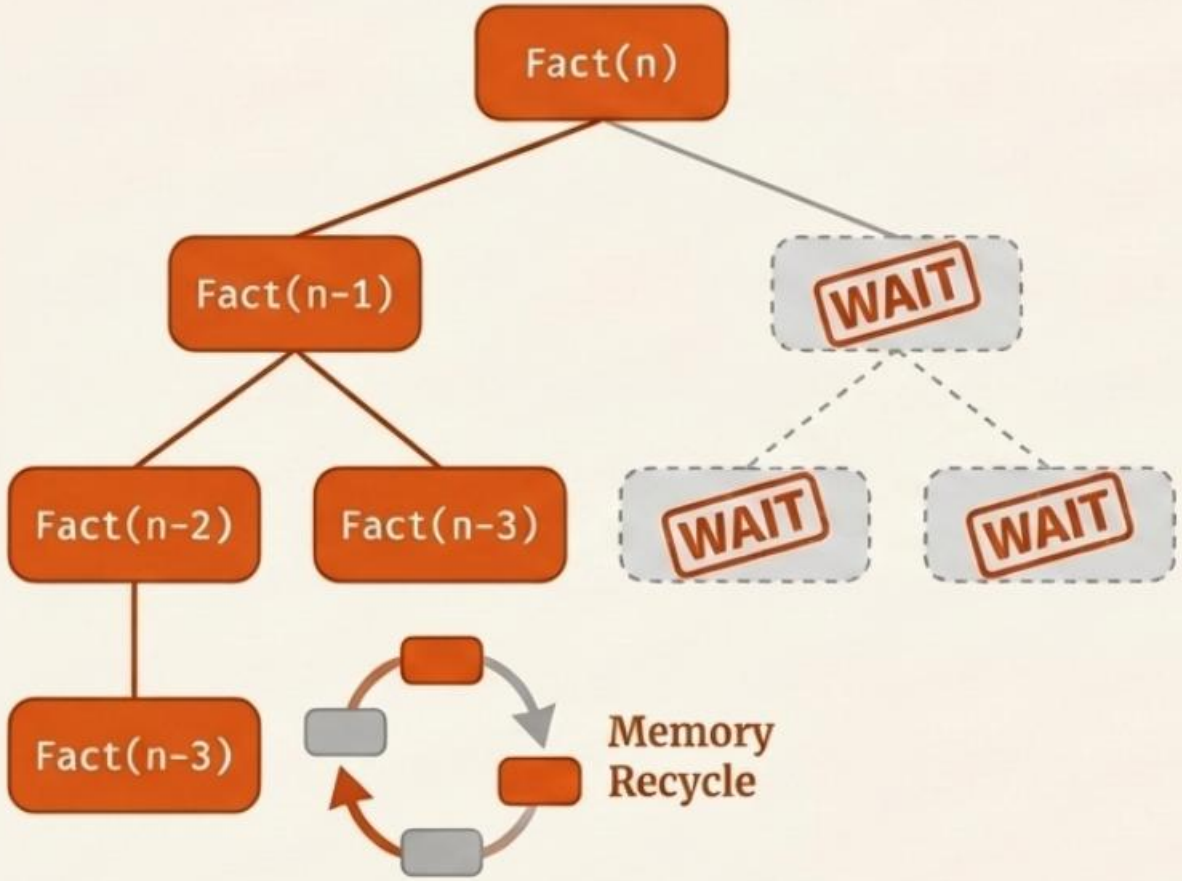
The computer tracks calls by pushing them onto a memory stack. The stack reaches its absolute maximum height right before the first return statement triggers the removal (popping) of variables.

Applying the Rule

Height: n levels deep.
Variables: 1 variable per level (int n).

Calculation: n (levels) × 1 (variable) = O(n) Space Complexity

Branching Recursion & The Wait Principle



The Wait Principle

A function calling itself twice creates an exponential tree. However, we do not store the whole tree at once. The left side fully executes while the right side waits.

Memory Recycling

Variables are deleted when a left branch finishes, freeing the exact space needed for the right branch to execute. The memory footprint never widens; it only goes as deep as the longest single path.

The Calculation

```
Maximum stack depth (n) × Variables per call (1)
= O(n) Space Complexity
```

Coding Example (Fibonacci):

```
int Fib(int n) {
  if (n <= 1) return n;
  int x = Fib(n-1);
  int y = Fib(n-2);
  return x + y;
}
```

Formula: Space Complexity = Height × Variables

Generalizing the Rule: Your Master Guide

Column 1: Finding Height

If the recursive call is **f(n-1)** -> The Height is **n** (Linear depth).

If the recursive call is **f (n/2)** -> The Height is **log n** (Logarithmic depth).

Column 2: Finding Variables

If the function only defines int **n** -> Variables = **1**.

If the function defines int **n**, int **i** -> Variables = **2**.

Coding Example: Linear Recursion with Two Variables

```
void exampleFunction(int n) {
    if (n <= 0) return;    // Base Case
    int x = 10;            // Variable 1
    int y = 20;            // Variable 2
    // n is also a variable passed in
    exampleFunction(n-1); // Recursive Call
}
```

Analysis:

- **Height:** The recursive call is f(n-1), so the depth is n (Linear).
- **Variables:** The function defines int n, int x, and int y in each call. Total = 3.
- **Calculation:** Space = Height (n) × Variables (3) = O(3n) = O(n) Space Complexity.

Total Space Complexity combines Input and Auxiliary memory

Total Space = Input Space + Auxiliary Space

Input Space

- Variables passed directly into the algorithm to define the problem (e.g., int n).
- The Rule: Generally ignored when comparing algorithms for the same problem, as the input size remains constant across all potential solutions.

Auxiliary Space

- Intermediate or temporary variables created internally to reach the solution.
- The Rule: This is the critical metric. When comparing algorithm efficiency, we evaluate the Auxiliary Space footprint.

Iteration solves the sum problem using strictly constant Auxiliary Space

Calculating the sum of all numbers from n down to 0.

Input Space: Ignored

```
function getSum(int n):
    int sum = 0
    int i = 0
    for i from 1 to n:
        sum = sum + i
    return sum
```

Auxiliary Space

Calculation: `S(n)` = 2 variables.

Result: `O(1)` Constant Space.

Memory usage remains fixed at 2 slots, even if n scales to one million.

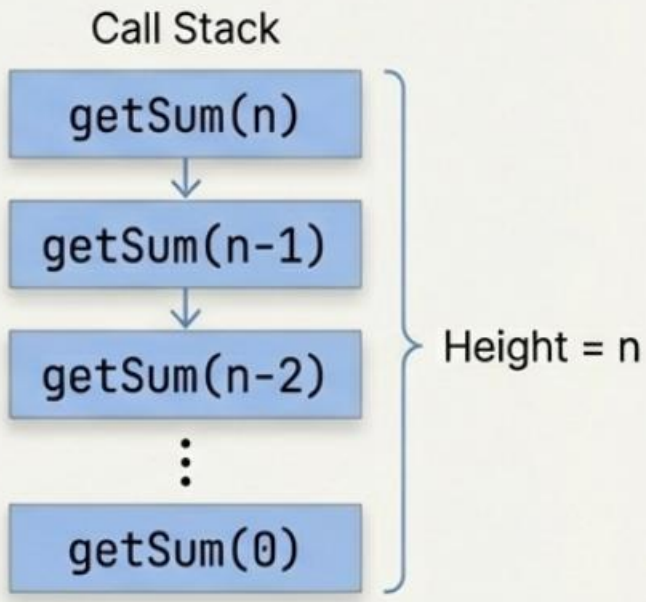
Recursion solves the same problem but consumes linear Auxiliary Space

Calculating the sum of all numbers from n down to 0.

Input Space (for initial call)

```
function getSum(int n):
  if n == 0: return 0
  return n + getSum(n - 1)
```

Auxiliary Space: Creates a new stack frame per call



Calculation: n recursive calls × 1 memory slot per new n variable. Total footprint = n.

Result: O(n) Linear Space.

Memory usage grows in direct proportion to the size of the input.

Evaluate both Time and Space before assuming recursion is the better choice

Metric	Iterative Algorithm	Recursive Algorithm
Time Complexity	`O(n) (Linear)`	`O(n) (Linear)`
Auxiliary Space	**`O(1) (Constant)`**	`O(n) (Linear)`

- Both approaches calculate the sum in the exact same amount of time.
- The recursive approach needlessly consumes scaling memory due to the call stack.

Consultant's Takeaway
Recursion is not a universal upgrade. Never rely on Time Complexity alone—always calculate Space Complexity to guarantee memory-efficient algorithms.

Questions

Find Space complexity?

```
int n = user_input;
for (int i = 0; i < n; i++) {
  // do something
}
```

```
int m = user_input;
int[] n = new int[m];
for (int i = 1; i <= m; i++) { print(n[i]); }
```

```
int Add(int n) {
  if (n <= 0) return 0;
  // Base case

  int variable = n;
  // 1 variable per
  level

  return n + Add(n - 1);
  // Recursive call
}
```

```
def pair_sum(a, b):
    x = a * 2   # Nested scope variable
    y = b + 1   # Nested scope variable
    return x + y

def add_sequence(n):
    sum = 0             # Base scope variable
    for i in range(n): # Base scope variable
        sum += pair_sum(i, i + 1)
    return sum

# Example call
add_sequence(10)
```

```
void exampleFunction(int n) {
    if (n <= 0) return;   // Base Case
    int x = 10;           // Variable 1
    int y = 20;           // Variable 2
    // n is also a variable passed in
    exampleFunction(n-1); // Recursive Call
}
```